

FIELD PROGRAMME RAHIM – WEEK 6

Predictive Maintenance System – Version I

By Friday, you should be able to:

- Understand the difference between detecting a current fault and predicting a future failure.
- Work with historical motor sensor data.
- Identify patterns that occur before a motor failure.
- Create meaningful features from time-series data.
- Prepare data for predictive Machine Learning.
- Train a model to predict potential motor failure.
- Evaluate the model using appropriate metrics.
- Test the model using previously unseen data.
- Save and reuse a trained Machine Learning model.
- Build a simple command-line predictive-maintenance system.

Day I – From Fault Detection to Failure Prediction

Learn

Review the difference between:

Fault Detection

"Is the motor faulty right now?"

Example:

Temperature: 85°C

Current: 11.2 A

RPM: 1300

Prediction: FAULT

Failure Prediction

"Based on what the motor is doing now, is it likely to fail in the future?"

For example:

10:00 → Normal
10:10 → Normal
10:20 → Temperature increasing
10:30 → Vibration increasing
10:40 → RPM decreasing
10:50 → Failure

The goal is to identify the **warning signs before the failure happens**.

Practical Work

Use a dataset containing historical motor/machine measurements and failure information.

If the dataset from Week 4 has failure records, continue using it.

Otherwise, use a suitable predictive-maintenance dataset.

Program 1 – Find Failure Cases

Explore the dataset and identify:

- How many machines/motors failed?
- How many did not fail?
- What percentage failed?
- Which sensor readings are present before failure?

Program 2 – Study a Failure

Choose one failure case.

Look at the readings leading up to it.

For example:

Time	Temperature	RPM	Current
10:00	60	1480	8.1
10:10	63	1460	8.4
10:20	67	1430	8.8
10:30	72	1390	9.4
10:40	78	1340	10.1
10:50	85	1280	11.2

Ask:

What changed before the failure?

Questions

Write your observations in:

failure_analysis.txt

Answer:

1. What happened before the failure?
 2. Which sensor changed the most?
 3. Were multiple sensors changing together?
 4. How much time passed between the first unusual reading and failure?
-

Git Task

Day 1 - Understanding Failure Prediction

Day 2 – Creating a Prediction Target

This is an important concept.

The dataset might tell us:

Failure = 1

But we don't just want to know whether the motor **already failed**.

We want to train a model that can recognize:

"This motor is approaching a failure."

Learn

Study:

- Target variable
- Prediction window
- Future information
- Data leakage
- Historical vs future data

Explain **data leakage** carefully.

For example, if we're trying to predict failure at 15:00, we shouldn't give the model information that only became available **after** 15:00.

Practical Work

Create a simplified prediction target.

For example:

Failure Soon

where:

0 = No failure soon

I = Failure occurs within the selected future window

The exact window depends on the dataset.

For this exercise, he can choose an appropriate window based on the available timestamps.

Example

Imagine:

10:00 → Normal
10:10 → Normal
10:20 → Normal
10:30 → Failure

If the prediction window is 30 minutes:

10:00 → Failure Soon = I
10:10 → Failure Soon = I
10:20 → Failure Soon = I

The idea is that the model learns:

"These sensor conditions occurred shortly before a failure."

Practical Task

Create:

`create_target.py`

The program should:

1. Load the motor dataset.
 2. Sort the data by time.
 3. Identify failure events.
 4. Create a `failure_soon` target.
 5. Save the processed dataset.
-

Deliverable

Create:

processed_motor_data.csv

with something similar to:

Temperature | RPM | Current | Vibration | Failure_Soon

60	1480	8.1	0.15	0
65	1450	8.5	0.20	0
72	1390	9.3	0.35	1
78	1340	10.1	0.50	1

Git Task

Commit and push.

Day 3 – Build the Predictive Model

Now we train the actual model.

Learn

Review:

- Features
- Target
- Training data
- Testing data
- Classification
- Random Forest
- Predictions

Practical Work

Use:

processed_motor_data.csv

Step 1 – Select Features

Choose relevant features such as:

Temperature
RPM
Current
Voltage
Torque
Vibration

Only use columns that actually exist in the dataset.

Step 2 – Select Target

Failure_Soon

Step 3 – Split the Data

Use training and testing data.

However, introduce an important concept:

Don't randomly mix future and past data when the data is time-dependent.

Instead, where the dataset supports it, use an earlier portion for training and a later portion for testing.

For example:

Historical Data

Training 80%	Testing 20%
Past	Future

This is closer to how the model would be used in reality.

Step 4 – Train

Use:

Random Forest Classifier

Step 5 – Predict

Generate predictions on the test data.

Step 6 – Evaluate

Calculate:

- Accuracy
- Precision
- Recall

- F1-score
 - Confusion Matrix
-

Important Question

Ask him:

For predictive maintenance, why might recall be particularly important?

He should start understanding that missing a real impending failure can be much more costly than generating an unnecessary warning.

Deliverable

Create:

`predictive_maintenance_model.py`

Git Task

Day 3 - Predictive Maintenance Model

Day 4 – Testing the Model & Understanding Mistakes

Today is about **not trusting the model blindly**.

Learn

Study:

- False positives
 - False negatives
 - Model limitations
 - Class imbalance
 - Thresholds
 - Precision vs recall
-

Practical Work

Program 1 – Analyze Predictions

Create a table containing:

Actual	Predicted

0	0
0	1
1	1
1	0

Count:

- True Positives
 - True Negatives
 - False Positives
 - False Negatives
-

Program 2 – Investigate False Negatives

Find examples where:

Actual = Failure Soon

but:

Prediction = Normal

Ask:

What did the sensor readings look like in these cases?

This is very important.

Program 3 – Investigate False Positives

Find examples where the model predicted:

Failure Soon

but the motor did not fail soon.

Ask:

Why might the model have thought there was a problem?

Program 4 – Feature Importance

Use the Random Forest model.

Identify which features were most useful.

For example:

Temperature	■	■	■	■	■	■
Vibration	■	■	■	■	■	■
Current	■	■	■	■	■	■
RPM	■	■	■	■	■	■
Voltage	■	■	■	■	■	■

Again:

Feature importance does not prove causation.

Questions

Write:

model_errors.txt

Answer:

1. How many false negatives did you have?
 2. How many false positives?
 3. Which type of error is more dangerous?
 4. Which sensor was most important?
 5. What could cause the model to make incorrect predictions?
-

Git Task

Commit and push.

Day 5 – Build the Predictive Maintenance System

Now turn everything into a small application.

Motor Predictive Maintenance System – Version I

The system should allow a user to enter current motor sensor readings.

For example:

```
MOTOR MONITORING SYSTEM
=====
```

Temperature: 74

RPM: 1390

Current: 9.7

Voltage: 414

Vibration: 0.42

The trained model should return something like:

```
RESULT
=====
```

Prediction: FAILURE LIKELY

Confidence: 82%

Recommendation:

Investigate the motor and inspect its operating conditions.

Or:

```
RESULT
=====
```

Prediction: NORMAL

Confidence: 91%

Recommendation:

No immediate warning from the model.
Continue monitoring.

Important

He should **not** write:

```
if temperature > 70:  
    failure = True
```

The prediction must come from the **trained Machine Learning model**.

Model Saving

Introduce:

joblib

Install:

```
pip install joblib
```

Teach him to:

Save the trained model

```
motor_model.pkl
```

Then:

Load the model later

So the application doesn't need to retrain every time it starts.

Final Project Structure

Something like:

```
week6/
├── data/
│   └── processed_motor_data.csv
├── notebooks/
│   └── predictive_maintenance.ipynb
├── src/
│   ├── create_target.py
│   ├── train_model.py
│   └── predict.py
├── model/
│   └── motor_model.pkl
├── README.md
└── requirements.txt
```

Don't worry if his implementation is slightly different. The **organization and understanding** matter more than the exact folder names.